AquaSentinel

AI-Powered Water Quality Monitoring & Early-Warning System

FINAL-YEAR B.TECH COMPUTER SCIENCE & ENGINEERING PROJECT

A full-stack intelligent monitoring platform that analyzes water-quality readings, detects unusual sensor patterns, estimates water-quality risk categories, and presents timely review alerts through a web dashboard.

Layer	Technology
Frontend	React + Vite
Backend	Java 17 + Spring Boot
AI/ML service	Python + FastAPI + Scikit-learn
Database	MySQL
Sensor integration	Optional ESP32 / Arduino; simulated data supported
Testing	JUnit + Pytest

Project report and implementation blueprint | Version 1.0 | 2026

Declaration and Important Scope

This report describes AquaSentinel, an academic prototype for analyzing water-quality measurements. It is designed for educational monitoring and decision-support demonstrations. It is not a certified laboratory instrument and must not be used to declare drinking water safe or unsafe without appropriate validated testing by qualified professionals.

Submission note: Add your name, roll number, department, college, supervisor, and academic year. Validate all implementation and test claims before submission. This PDF is the report and technical blueprint; the executable React, Java, Python, SQL, and configuration files must be committed separately to GitHub.

Abstract

AquaSentinel is a full-stack platform for collecting and analyzing water-quality readings from sensors or uploaded datasets. It combines a React dashboard, Java Spring Boot backend, Python machine-learning service, and MySQL database. The platform stores readings such as pH, turbidity, temperature, and electrical conductivity, checks values against configured reference ranges, and applies anomaly detection to highlight unusual patterns. It provides visual trends, sensor status, and review alerts. The system is intended as a prototype for monitoring workflows and does not replace certified water testing.

Keywords: Water quality, IoT, anomaly detection, machine learning, sensor analytics, Spring Boot, FastAPI, React, MySQL.

1. Introduction

Water monitoring programs may collect readings from multiple locations and devices. Reviewing measurements over time can be difficult when readings are scattered across spreadsheets or individual devices. AquaSentinel demonstrates how a centralized platform can organize measurements, identify unusual changes, and help users decide when additional verification may be needed.

1.1 Problem statement

Develop an accessible, extensible platform that receives water-quality measurements, detects abnormal patterns, visualizes historical trends, and raises review alerts while communicating the limitations of low-cost sensors and predictive models.

1.2 Objectives

• Provide a dashboard for current and historical water-quality readings.

• Accept readings from simulated sensors, CSV upload, or authorized IoT devices.

• Build Java REST APIs for ingestion, validation, storage, and alert management.

• Use Python ML to detect unusual multi-parameter patterns.

• Store readings, device metadata, and alerts in MySQL.

• Provide reproducible tests, documentation, and GitHub deployment instructions.

1.3 Scope

In scope: prototype sensor-data ingestion, time-series visualization, anomaly detection, rule-based threshold alerts, and human review workflows. **Out of scope:** certification of water safety, autonomous public-health decisions, or replacing laboratory analysis.

2. System Architecture

A React client displays readings and alerts. The Java backend validates incoming data, persists it in MySQL, and calls the Python service for anomaly scoring. The ML service applies preprocessing and returns a score with model metadata. The backend combines model output with configurable parameter thresholds and exposes results to the dashboard.

Layer	Technology	Responsibilities
Presentation	React + Vite	Charts, sensor list, filters, alert details
Application	Java 17 + Spring Boot	REST APIs, validation, persistence, workflow

Layer	Technology	Responsibilities
ML service	Python + FastAPI + Scikit-learn	Preprocessing, anomaly detection, scoring
Data	MySQL	Devices, readings, alerts, model versions
Edge (optional)	ESP32 / Arduino	Collect readings and send authenticated payloads

2.1 Data flow

• A sensor simulator, CSV import, or authorized device submits a measurement.

• Spring Boot validates units, timestamp, device identity, and numeric ranges.

• The reading is stored in MySQL.

• The backend sends selected normalized features to the Python model service.

• The model returns an anomaly score and model version.

• Configured threshold rules and the anomaly score create a review alert when warranted.

• The dashboard displays trends and alerts; a user records review status.

2.2 Suggested repository structure

```
aquasentinel/
  frontend/                 React + Vite dashboard
  backend/                  Java 17 + Spring Boot API
  ml-service/               Python + FastAPI + Scikit-learn
    app/
    tests/
    data/
  simulator/                Synthetic sensor data generator
  database/schema.sql
  docs/
    AquaSentinel_Project_Report.pdf
    API_SPECIFICATION.md
    PRESENTATION_OUTLINE.md
  docker-compose.yml
  README.md
  .gitignore
```

3. Functional Requirements

ID	Requirement	Expected behavior
FR-01	Device registration	Register a demo sensor/device
FR-02	Reading ingestion	Accept validated measurement payload
FR-03	History	Retrieve readings by device and date range
FR-04	Anomaly detection	Return score and model version
FR-05	Threshold alerts	Flag readings outside configured ranges
FR-06	Alert review	Open, investigating, resolved states
FR-07	Dashboard	Show trends, latest readings, and alert counts
FR-08	CSV import	Validate and ingest a permitted dataset

3.1 Non-functional requirements

Reliability: handle missing sensor values and service outages. Performance: paginate history and batch imports. Security: authenticate device submissions, protect secrets, restrict CORS, and validate payload size. Maintainability: separate ingestion, ML, and presentation. Privacy: do not collect unnecessary personal information. Traceability: store timestamps, device identifiers, and model versions.

4. Data and Machine-Learning Design

The prototype can use Isolation Forest to identify unusual combinations of measurements. A separate rule-based layer checks configured measurement ranges. A supervised model may be added only when a properly labeled, representative dataset exists.

4.1 Example input parameters

Parameter	Meaning	Unit / note
pH	Acidity/alkalinity indicator	Dimensionless
turbidity	Cloudiness indicator	NTU; sensor calibration required
temperature	Water temperature	°C
conductivity	Electrical conductivity	µS/cm; temperature compensation matters
total_dissolved_solids	Estimated dissolved solids	mg/L; often derived from conductivity

Use parameter units consistently and record calibration metadata. A low-cost sensor's readings can drift or be affected by installation, temperature, and maintenance.

4.2 ML workflow

• Collect authorized historical readings or generate synthetic demo data.

• Validate units, remove impossible values, and document missing-data handling.

• Create a chronological train/validation/test split to reduce time leakage.

• Fit preprocessing and anomaly model on training data only.

• Select alert thresholds using validation data and domain guidance.

• Evaluate on a held-out period and document false positives and missed anomalies.

• Version the model, feature schema, dataset provenance, and evaluation results.

4.3 Evaluation

For labeled anomaly data, report precision, recall, F1-score, confusion matrix, and PR-AUC. For unlabeled data, use expert-reviewed samples and report alert volume, review yield, and stability over time. Do not invent accuracy values. An anomaly score is not a probability that water is contaminated.

4.4 Alert categories

Category	Trigger	Response
Parameter threshold	Configured reference range exceeded	Request confirmatory measurement
Unusual pattern	ML model flags multi-feature deviation	Review recent history and sensor health
Sensor quality	Missing, stale, or implausible reading	Inspect calibration/device status

Reference ranges depend on the intended use, jurisdiction, sampling method, and applicable standards. Store configurable reference limits with their source and version; do not hard-code one universal 'safe water' threshold.

5. Database Design

Starter MySQL schema for the prototype:

```
CREATE DATABASE IF NOT EXISTS aquasentinel;
USE aquasentinel;

CREATE TABLE devices (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  device_code VARCHAR(80) NOT NULL UNIQUE,
```

```
  display_name VARCHAR(120) NOT NULL,
  location_label VARCHAR(120),
  status VARCHAR(24) NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE readings (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  device_id BIGINT NOT NULL,
  ph DOUBLE,
  turbidity_ntu DOUBLE,
  temperature_c DOUBLE,
  conductivity_us_cm DOUBLE,
  tds_mg_l DOUBLE,
  measured_at TIMESTAMP NOT NULL,
  received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (device_id) REFERENCES devices(id),
  INDEX idx_reading_time (measured_at)
);

CREATE TABLE alerts (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  reading_id BIGINT NOT NULL,
  alert_type VARCHAR(60) NOT NULL,
  anomaly_score DOUBLE,
  severity VARCHAR(20) NOT NULL,
  status VARCHAR(24) NOT NULL DEFAULT 'OPEN',
  message VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (reading_id) REFERENCES readings(id)
);

CREATE TABLE model_versions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  model_name VARCHAR(100) NOT NULL,
  version VARCHAR(40) NOT NULL,
  dataset_version VARCHAR(80),
  evaluation_summary VARCHAR(1000),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

For production, add device authentication, secret rotation, migrations, retention rules, and indexes based on actual query patterns. Avoid storing exact private addresses in location_label.

6. API Specification

Method	Endpoint	Purpose
POST	/api/devices	Register a demo device
GET	/api/devices	List devices
POST	/api/readings	Submit a reading
POST	/api/readings/import	Import validated CSV
GET	/api/readings	Query readings by device/date
GET	/api/alerts	List alerts
PATCH	/api/alerts/{id}/status	Update review status
GET	/api/dashboard/summary	Dashboard summary
POST	/predict	ML anomaly inference
GET	/ml/health	ML service health

6.1 Example reading payload

```
{
  "deviceCode": "AQUA-DEMO-01",
  "ph": 7.2,
```

```
  "turbidityNtu": 1.8,
  "temperatureC": 24.5,
  "conductivityUsCm": 420,
  "tdsMgL": 270,
  "measuredAt": "2026-11-03T08:00:00"
}
```

6.2 Example ML response

```
{
  "anomalyScore": 0.76,
  "anomalyFlag": true,
  "modelName": "IsolationForest",
  "modelVersion": "0.1.0",
  "message": "Unusual combination of sensor readings; verify measurement."
}
```

The sample response is illustrative only. It is not a water-safety conclusion.

7. Frontend Design

Recommended dashboard screens: overview cards for active devices and open alerts; time-series charts for each parameter; device health and last-seen time; reading table with date/device filters; alert detail view with sensor values and model version; and an analyst review workflow. Include accessible labels, clear units, loading states, and stale-data indicators.

Screen	Elements
Overview	Latest readings, device health, alert counts
Device details	Parameter trends, last received timestamp
Readings	Search, filters, pagination, CSV export
Alert details	Trigger, values, threshold source, model score, status
Model information	Model version, dataset provenance, evaluation metrics

8. Testing Strategy

Area	Test case	Expected result
Backend validation	Out-of-range numeric value	HTTP 400 with field error
Ingestion	Valid sensor reading	Persisted with timestamp
Database	Reading linked to device	Foreign key integrity
ML unit	Valid feature vector	Finite anomaly score
ML validation	Missing required fields	Safe 4xx response
Integration	ML service unavailable	Reading retained; analysis marked pending/error
Frontend	Stale sensor reading	Visible stale-data indicator
End-to-end	Submit reading and review alert	Alert displayed and status persists

Execute the tests and attach actual logs/screenshots before claiming pass status.

9. Local Setup Plan

Prerequisites: Java 17+, Maven 3.8+, Python 3.10+, Node.js 18+, MySQL 8+, and Git.

```
# Database
mysql -u root -p < database/schema.sql

# Python ML service
```

```
cd ml-service
python -m venv .venv
source .venv/bin/activate
# Windows PowerShell: .venv\\Scripts\\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8001

# Java backend (new terminal)
cd backend
./mvnw spring-boot:run

# React frontend (new terminal)
cd frontend
npm install
npm run dev
```

Suggested ports: React 5173, Spring Boot 8080, FastAPI 8001. Confirm actual settings in the application. If no physical sensor is available, use the included simulator or a documented synthetic CSV dataset.

10. Safety, Validation, and Responsible Use

• Do not use the dashboard as a substitute for certified laboratory water testing.

• Never describe a reading as safe solely because the ML model did not flag it.

• Calibrate sensors and document sensor model, calibration date, units, and uncertainty.

• Use authorized sensor data; avoid collecting unnecessary personal or exact location information.

• Keep device credentials and database passwords out of GitHub.

• Require human verification before communicating public-health conclusions.

• Record the reference standard and version used for any threshold configuration.

11. Limitations and Future Enhancements

Limitations include sensor drift, calibration errors, missing readings, environmental variation, dataset bias, and false alerts. Future enhancements may include secure MQTT ingestion, device provisioning, calibration reminders, explainability, model drift monitoring, offline edge inference, role-based access, Docker deployment, and a CI pipeline.

12. Conclusion

AquaSentinel demonstrates an end-to-end IoT and AI/ML monitoring architecture using React, Java Spring Boot, Python, and MySQL. It is suitable for a final-year CSE prototype when accompanied by working code, documented sensor assumptions, reproducible tests, honest model evaluation, and clear limits on interpretation.

13. Presentation Outline

Slide	Title	Content
1	Title	Project name and student details
2	Problem	Fragmented monitoring and changing readings
3	Objectives	Monitoring, anomaly detection, alerts
4	Architecture	React, Spring Boot, FastAPI, MySQL, sensors
5	Data flow	Sensor/simulator to dashboard
6	ML design	Features, Isolation Forest, evaluation
7	Database and APIs	Schema and key endpoints
8	Demo	Submit synthetic reading, view trend and alert

Slide	Title	Content
9	Testing	Actual test evidence and metrics
10	Limitations	Calibration, false alerts, lab verification
11	Future scope	MQTT, edge inference, model monitoring
12	Q&A	Questions

14. GitHub Checklist and Push Commands

■ Commit frontend/, backend/, ml-service/, simulator/, database/, docs/, and README.md.

■ Include only synthetic demonstration data or data with clear permission/licensing.

■ Add screenshots and verified test outputs.

■ Exclude node_modules, target, .venv, .env, logs, secrets, and private data.

■ Document exact run commands and tested software versions.

```
git init
git add .
git commit -m "Initial commit: AquaSentinel CSE project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/aquasentinel.git
git push -u origin main

# Later updates
git add .
git commit -m "Add sensor analytics dashboard"
git push
```

Before pushing: Create an empty GitHub repository named aquasentinel and replace YOUR-USERNAME. Never include passwords or access tokens in source files or remote URLs.

References

• Spring Boot: https://docs.spring.io/spring-boot/

• Scikit-learn: https://scikit-learn.org/stable/

• FastAPI: https://fastapi.tiangolo.com/

• React: https://react.dev/

• MySQL: https://dev.mysql.com/doc/

• WHO drinking-water guidance: https://www.who.int/teams/environment-climate-change-and-health/water-sanitation-and-health/water-safety-and-quality/drinking-water-quality-guidelines